

Architecture and trade-offs

This document describes what the ledger, as built, would meet in production. It stands on its own: everything it needs is explained in it. It covers four questions. What grows without bound at scale, and the cheapest fix. What value-dated entries do to a bank's operations, and one control to add. Every way an authorization can end. And what was left out, with the risk each cut defers.

How the ledger works

The ledger is a list of **entries**. An entry is one line: which account, how much (a credit is positive, a debit is negative), the **booking day** (the day the ledger wrote it) and the **value day** (the day the money counts for). Entries are only ever added at the end of the list. None is ever changed or removed.

The **closing balance** of an account for a day is the sum of its entries whose value day is on or before that day. It is not stored anywhere. Each time it is needed, the ledger reads the account's entries from the start and adds up the ones that count. So the balance for any past day can always be asked for, and the answer reflects every entry known so far.

A **back-dated entry** is one whose value day is earlier than its booking day: money that counts for a day that has already ended. When one arrives, the closing balances of every day from its value day onward change, because those days are recomputed from the entries.

A **hold** is money set aside when a card authorization is approved. It is not an entry. It lowers what the account may spend (the **available balance**: closing balance of today minus open holds) but not the closing balance. A hold is checked once, when the authorization arrives, and is then open until it is settled.

At the end of each day, after the day's last event, the ledger runs the **nightly run**: six steps in a fixed order.

1. **Cutoff.** The day is closed. Nothing more can be booked on it.
2. **Re-evaluate.** For each account and each earlier day, compare the closing balance now with the closing balance the previous nightly run recorded. List the days that moved. This is how a back-dated entry shows up.
3. **Fees.** For each account, walk the days from the first to today, in order. A day whose closing balance is below zero and that has no fee yet gets one fee entry, with that day as its value day. Walking in order matters: the fee for one day lowers every later day.
4. **Interest.** For each account and each day up to today, work out that day's interest on its closing balance. This is a side figure, kept in the run's output. No entry is written.
5. **Capitalize.** On the last day only, write one entry per account for the total of the daily interest figures.
6. **Report.** Record each account's closing balance for each day, so the next nightly run can compare against it in step 2.

Steps 2, 3, 4 and 6 each read the closing balance of every past day of every account. Step 3 is the only one that writes during the walk. Because every balance is recomputed on request, a day has two figures worth telling apart: **as closed**, the balance when that day's nightly run finished, and **final**, the balance from everything known now. They differ wherever a later entry was dated back onto that day.

1. Append-only at scale

What breaks first: the nightly run's time, not memory

Every closing balance is a scan of the account's entries. The nightly run asks for the closing balance of every past day, for every account, in four of its six steps. Each ask is a fresh scan; nothing is cached between steps.

So one nightly run costs about: number of accounts $\times$ number of days so far $\times$ entries per account, four times over. The "days so far" factor is what hurts. On day 10 the run looks back over 10 days; on day 365 over 365. Across a whole window, the total work grows with the square of the number of days: doubling the window quadruples the work.

At 100 $\times$ the entries and a year of days, the nightly run is the first thing that stops fitting in the time between one day's close and the next day's opening. Memory is not the limit. An entry is a handful of numbers, and 100 $\times$ more of them still fit in a single machine long after the run has become too slow.

Where state grows without bound

Four things only ever grow, and each exists for a reason:

- **The list of entries.** Never pruned. That is what append-only means.
- **The holds.** Every hold ever opened is kept, including the ones that were declined or settled long ago, so the record of each authorization survives.
- **The ids already seen.** Every event id that was applied, and which event reversed which. Needed to refuse a duplicate event or a second reversal of the same event.
- **The recorded closings.** One figure per account per day, written by step 6 so step 2 can report what changed.

The cheapest fix, in two steps

Step one: cache the closing balances until the next entry is added. The only thing that changes a closing balance is a new entry. So a table of closings, one per account per day, is correct until the next append, and can be thrown away then. Inside one nightly run only step 3 appends (it writes fees as it walks). So step 2 can use one scan, step 3 refreshes the cache after each fee it writes, and steps 4 and 6 share one scan taken after the fees. Four scans per day become about two. No rule changes.

Step two: keep the closings up to date instead of throwing them away. When an entry is added, add its amount to the stored closing of its value day and of every day after it. Then a closing balance is a lookup, not a scan, and the list of entries is something that is appended to and read rarely. Add a **back-dating limit**: an entry may have a value day at most N days before its booking day. With that, a new entry can only touch the last N stored closings, so the work per entry is fixed and does not grow with history.

This is cheaper than moving to a database or splitting accounts across machines because it changes no rule and no interface. It only changes where a number is kept. The list of entries stays the source of truth; the stored closings can be rebuilt from it at any time. That is the difference between this and a running balance that is updated in place and is the only record: here the balance is a copy you may discard.

What this does not fix: the holds still grow, because nothing ever expires one (the fix, an expiry by card-scheme rules, is under [Authorization lifecycle](#)), and the ids already seen still need to be archived once events are old enough that a duplicate can no longer arrive.

2. Value-dated entries in production

What a value date does to a bank

Day 2 ends. The bank has shown the customer a balance for Day 2, used it for the day's interest, carried it into its own books, and will report figures built on it. Three days later an entry arrives that counts for Day 2. Now Day 2 has a new balance. Every figure that was built on the old one is wrong, and some of those figures have already left the building.

This ledger handles that by recomputing: the fees and the interest for Day 2 are worked out again from the new balance. It can do that because nothing has been paid out yet inside the window. Outside a six-day window, that stops being true.

Three places it bites

- **The customer.** A statement already sent, a fee already charged, interest already credited. Once interest has been paid out, a back-dated entry needs an **adjustment entry**, a credit or a debit for the difference. For a large balance or a long back-dating that difference is not small, and a debit adjustment can push an account below zero when the customer has already spent the money.
- **The bank's books.** A closed accounting period is not rewritten. A correction is a new entry in the open period that says what it corrects. The two figures this ledger keeps for each day, as closed and final, are the in-memory version of that rule: both are kept, neither is overwritten, and step 2 of the nightly run prints what moved.
- **The regulator.** The Central Bank of the UAE's Consumer Protection Regulation (Circular No. 8/2020) and its Consumer Protection Standards (Notice No. 1158/2021) require fees to be disclosed and agreed in advance, give customers at least 60 calendar days' notice before a fee changes (clause 2.1.1.47), and, where a deduction is caused by the bank's own error, require the error to be corrected and the amount refunded at once, with the customer told within ten business days (clauses 5.1.1.38 to 5.1.1.40). A fee that appears on a day three days after it ended, caused by someone else's late posting, is exactly the case that produces complaints and refunds

under those clauses. That is why this ledger does not refund such a fee on its own and does not charge it silently: when a later reversal leaves a fee behind on a day that is now positive, the fee is flagged for a person to decide. Source: the Central Bank's rulebook, <https://rulebook.centralbank.ae/en/rulebook/consumer-protection-standards> (read 7 October 2026).

The one control to add before going live

A back-dating limit, with maker-checker beyond it. An entry may be value-dated at most N days back, where N is set by policy (30 is a common choice; the design works for any N). Anything older is refused by the ledger. It can still be booked, but only as an adjustment entry in the current period, and only through **maker-checker**: one person prepares it, a different person approves it, and the reason is recorded with it.

This control rather than another because it bounds every problem in this section at once: how large an interest adjustment can be, how many days a nightly run must re-evaluate, and how old a statement can be invalidated. It is also the same limit that the scale fix needs: with it, a new entry can only change a fixed number of stored closing balances (see [The cheapest fix, in two steps](#)).

3. Authorization lifecycle

An **authorization** is a card payment request. The ledger approves it if the available balance stays at or above zero after setting the amount aside, and then keeps a hold for that amount. A **settlement** is the merchant's later claim for the money: it is accepted against an open hold for up to the hold's amount, the debit is written, and the hold is closed. In this ledger a hold has three states: open, settled, declined.

Other than a settlement for the full amount, an authorization can end in four ways.

How it ends	What happened in the real world	What the system must do
Declined on arrival. The available balance would go below zero.	A card is used with no funds behind it.	Record the decline and the balance it saw. Change nothing. Tell the card processor at once. Keep the record for disputes and fraud checks.
Settled for less than the hold. The difference is released.	The final amount is below the pre-authorized one: a fuel pump, a restaurant bill without the tip, a partial shipment.	Release the difference the moment the settlement is written, and close the hold. Never leave a sliver held.
Settlement refused. No such authorization, wrong account, already settled, or more than the hold.	A clearing message with a bad reference, a duplicate clearing file, or an over-claim.	Refuse it, write nothing, raise an exception for operations. The authorization keeps whatever state it had. Gap: card schemes allow a settlement with no prior authorization (a force-post) and over-claims within a tolerance. Both are left out of this ledger; the risk of each is listed under What was cut and why .
Never settled. The window ends with the hold still open.	The merchant never claims (a cancelled order, a failed shipment), or claims after the scheme's deadline.	Not handled: the hold stays open and lowers the available balance for ever. Required: an expiry by scheme rules (days for most merchants, longer for hotels and car rental) that releases the money, and a merchant-initiated reversal that releases it early.

Two more ends have no event in this ledger at all: an **incremental authorization**, where the merchant raises the held amount, and a **merchant reversal**, where the merchant cancels the authorization. Each is one more state change on the same hold record, not a new concept.

4. What was cut and why

Every simplification made to stay in scope, with the production risk it defers. In the order a reader meets them.

Cut	Why	Risk deferred
A fee left behind by a reversal is flagged, never refunded by the ledger	The ledger writes only what an event or a rule caused. A refund is a policy decision.	Fees a customer sees as unfair sit until someone acts. Needs a review queue with a deadline.
No settlement without a prior authorization (no force-post)	The ledger cannot verify a claim it has no record of.	Real claims are refused. Needs a force-post path with its own limits and reporting.
One settlement per authorization, never more than the hold	The event stream has one claim per authorization.	Several claims against one hold, and over-claims within a tolerance, are refused. Needs a partial-settlement state and a tolerance rule.
No hold expiry	No event in the stream expires a hold.	Money stays held for ever on an authorization the merchant never settles, lowering what the customer can spend. The fix is an expiry by card-scheme rules, described under Authorization lifecycle .
No back-dating limit	The window is six days.	An entry can reach back any number of days, so a nightly run may have to recompute all of history, and an interest adjustment after a back-dated entry can be any size. The limit is the first control to add, described under The one control to add before going live , and it is what makes the stored-closings fix work (see The cheapest fix, in two steps).
The overdraft fee is defined per currency, with no conversion	Only one currency has a fee. Converting would invent a number.	An account in a currency with no fee that closes below zero is an error, not a fee. The deliberate failing test shows it. Needs a fee schedule per currency.
Interest recomputed freely, with no adjustment entries	Nothing is paid out before the last day.	After interest is paid, a back-dated entry needs an adjustment entry with its own value day and an approval.
Closing balances scanned from the entries, never stored	Correct and simple at ten events.	Each nightly run rescans every entry of every account for every past day, so its time grows with the square of the number of days. The fix, cached and then stored closing balances, is described under The cheapest fix, in two steps .
A duplicate event id is the only duplicate check	Enough for one replay of one stream.	The same event delivered again under a new id is written twice. Needs a content hash or a sequence number from the sender.
One process, nothing saved to disk, one event at a time	Out of scope by the task's own terms.	Nothing survives a restart. Two writers on one account would race. Needs a durable log and one writer per account at a time.
Single-entry: each entry names one account, with no counterparty	One account per event is all the stream needs.	There is no second side whose total must balance the first, so a wrong sign only shows when a test asks. Production would make each entry one side of a double-entry posting: the fee is income to the bank, interest is an expense, a settlement goes to a clearing account.

The cuts are not independent. Hold expiry, the back-dating limit and the stored closings are one control seen from three sides: a bound on how far back a change may reach and how long state may live. That is the one to build first.